INSTITUTO PROFESIONAL SAN SEBASTIÁN

Digitalización inicial de solicitudes de venta Hielos del Maipo

Informe técnico de la Unidad 1

Asignatura: Desarrollo Backend (IF201IINF)

Docente: [Completar nombre del docente]

Integrantes: [Completar nombres y apellidos]

Fecha: 27 de octubre de 2026

Entrega: Evaluación Sumativa U1

Documento de trabajo: los resultados de ejecución local y las capturas de evidencia deben completarse antes de la entrega.

1. Contexto y alcance

Hielos del Maipo realiza ventas de hielo al por mayor y menor. En su ficha de vinculación con el medio, la empresa informa dificultades de organización, planificación de rutas y registro de ventas. Esta Unidad 1 desarrolla una primera API local para registrar solicitudes de pedido. El módulo es una propuesta académica y no significa que las reglas o campos hayan sido confirmados por la empresa.

Alcance: una entidad SolicitudPedido, seis rutas HTTP, validaciones sencillas, persistencia SQLite y un listado paginado. No se realizan cobros, control de stock, distribución, autenticación completa ni tienda en línea.

2. Arquitectura cliente-servidor

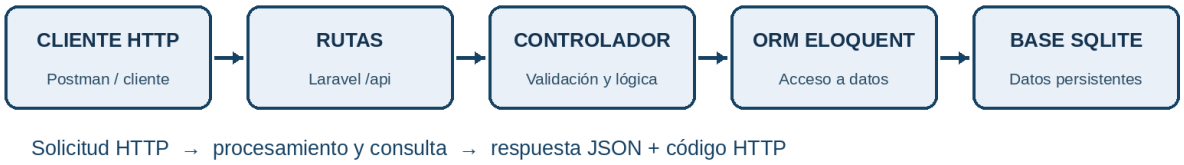


Figura 1. Flujo lógico del prototipo. El cliente envía una petición HTTP; Laravel identifica la ruta y ejecuta el controlador. Las clases FormRequest validan los datos; el modelo Eloquent realiza consultas sobre SQLite. El servidor devuelve JSON y un código HTTP según el resultado.

Componente	Responsabilidad en este proyecto
Cliente HTTP	Envía GET, POST, PUT, PATCH o DELETE desde Postman.
Rutas y servidor Laravel	Recibe las solicitudes, asigna los endpoints y dirige al controlador.
Controlador y FormRequest	Verifica datos de entrada, coordina CRUD y construye respuestas JSON.
Eloquent ORM	Consulta y modifica registros sin concatenar SQL manualmente.
SQLite	Guarda la tabla en un archivo local para que los registros persistan.

Se eligió una arquitectura sencilla y modular. Separar las rutas, la validación, el controlador y el modelo facilita el mantenimiento y permite añadir nuevas funcionalidades más adelante. Se mejora la robustez al tratar errores de validación y registros inexistentes con respuestas HTTP específicas. La paginación es la medida inicial de escalabilidad dentro del alcance de la unidad.

3. Modelo de datos y persistencia

Base de datos: SQLite. Tabla: solicitud_pedidos. Modelo ORM: App\Models\SolicitudPedido. La migración define los siguientes campos:

Campo	Tipo	Uso
id	Identificador entero	Clave primaria autoincremental
cliente_referencia	Texto (120)	Referencia ficticia del cliente
cantidad	Entero positivo	Cantidad de prueba; unidad por validar
fecha_requerida	Fecha	Valor AAAA-MM-DD
estado	Texto (20)	pendiente / en_revision / finalizada
created_at / updated_at	Fecha y hora	Registro técnico de creación y actualización

La persistencia se consigue con una migración real y operaciones Eloquent create, consulta, update y delete. Las variables de entorno permiten configurar la conexión fuera del código fuente. El archivo .env y el archivo SQLite no deben subirse al repositorio.

4. API REST: endpoints y respuestas

Método	Ruta	Uso / respuesta esperada
GET	/api/solicitudes	Lista paginada. 200 OK.
GET	/api/solicitudes/{id}	Consulta un registro. 200 OK o 404.
POST	/api/solicitudes	Crea un registro. 201 Created o 422.
PUT	/api/solicitudes/{id}	Reemplaza datos. 200, 404 o 422.
PATCH	/api/solicitudes/{id}	Modifica campos indicados. 200, 404 o 422.
DELETE	/api/solicitudes/{id}	Elimina un registro. 204 o 404.

Petición de ejemplo (datos ficticios): {"cliente_referencia":"Cliente de prueba 01","cantidad":8,"fecha_requerida":"2026-11-15","estado":"pendiente"}. Al crear se espera HTTP 201 y JSON con el identificador real generado por SQLite. Si una cantidad es negativa o un estado no está admitido, Laravel responde HTTP 422 con errores por campo. Para un identificador inexistente, corresponde 404.

La colección Postman adjunta incluye los casos de creación, consulta, edición completa y parcial, eliminación, errores de validación y registro inexistente. Las salidas del cliente deben comprobarse y adjuntarse tras ejecutar el servidor.

5. Lógica de negocio y calidad

Reglas del prototipo: referencia del cliente obligatoria y de máximo 120 caracteres, cantidad entera mayor que cero, fecha obligatoria con formato AAAA-MM-DD, y estado perteneciente a pendiente, en_revision o finalizada. Son supuestos académicos: no se afirma que estos sean los estados o unidades utilizados por Hielos del Maipo.

Lista de verificación de calidad

Criterio objetivo	Revisión en el código	Resultado / evidencia real
Organización	Rutas, controlador, validaciones y modelo separados.	Revisión estática completada; ejecución por comprobar.
Respuestas correctas	201 / 200 / 204 / 404 / 422 definidos.	Pendiente: capturas Postman.
Validación	Reglas en FormRequest; casos inválidos en tests.	Pendiente: resultados HTTP.
Persistencia	Migración SQLite y CRUD mediante Eloquent.	Pendiente: reinicio y GET repetido.
Consultas acotadas	Paginación por defecto de 10 y máximo de 50.	Pendiente: medición comparativa.

Mejora concreta incorporada: el método de listado utiliza paginate() y un máximo de 50 resultados por petición. También se organizó la validación en clases separadas para mantener mensajes y reglas consistentes. La sintaxis de los archivos PHP entregados fue revisada con php -l; esto no reemplaza la ejecución del framework.

6. Rendimiento y escalabilidad

Problema reproducible: sin límite de resultados, una consulta que recupera todas las solicitudes puede aumentar innecesariamente la cantidad de información enviada al cliente. La propuesta consulta 10 registros por página y acepta hasta 50. Para probarlo se incluye un seeder de 60 registros ficticios y un script que compara una consulta Eloquent sin paginación con otra que obtiene la primera página.

Indicador	Antes: sin paginar	Después: página de 10
Filas recuperadas	[Medir en terminal]	[Medir en terminal]
Tiempo observado (ms)	[Medir en terminal]	[Medir en terminal]
Interpretación	Todas las filas	Solo registros de una página

Registrar los valores reales utilizando php artisan db:seed --class=SolicitudPedidoSeeder y php scripts/comparar_paginacion.php. La comparación de cantidad de filas es reproducible; los tiempos en un equipo local no prueban por sí solos mayor rendimiento bajo carga. A futuro, podrían evaluarse índices, monitoreo, caché y nuevas capas según aumente el volumen real.

7. Seguridad conceptual

Autenticación: verifica quién es el usuario que accede al sistema. Autorización: define qué acciones puede realizar cada usuario una vez identificado. SSL/TLS: protege la comunicación en tránsito, generalmente mediante HTTPS. Son conceptos diferentes y ninguno sustituye a los demás.

Esta primera versión se limita a localhost y no incorpora control de acceso productivo, porque la pauta no lo exige aún. El ORM ofrece consultas parametrizadas, los datos de ejemplo son ficticios y las credenciales o claves de Laravel se mantienen en variables de entorno excluidas de Git. No se debe publicar esta API abierta en Internet.

8. Registro de pruebas y continuidad

Evidencias pendientes de completar en el equipo local: captura POST 201, GET 200, PUT/PATCH 200, DELETE 204, solicitud inválida 422 y GET 404; imagen del esquema o migración; terminal con pruebas automáticas; confirmación de que el mismo registro se conserva tras reiniciar php artisan serve; comparación real de consulta paginada y sin paginar.

Preguntas para la empresa: unidad de medida de los productos, identificación de clientes, estados de solicitud reales, fechas a registrar, permisos de usuarios y vínculo futuro con producción y reparto. La ampliación hacia inventario, costos o rutas necesita levantamiento adicional.

9. Declaración de uso de IA

Se utilizó ChatGPT como apoyo para analizar la pauta y la rúbrica, estructurar el informe técnico y elaborar una propuesta de código, pruebas y documentación. Antes de entregar, los estudiantes deben revisar el material, ejecutar la solución y documentar los resultados obtenidos personalmente. Esta declaración se incluye para cumplir con la instrucción de transparencia de la asignatura.

Fuentes consultadas

Instituto Profesional San Sebastián. Evaluación Sumativa U1 - Desarrollo Backend (IF201IINF); Rúbrica Evaluación Sumativa U1; Ficha VcM Actor Externo - Hielos del Maipo; Cuaderno de Ejercitación U1 y sesión práctica S3. Documentación oficial Laravel 12: https://laravel.com/docs/12.x/installation y https://laravel.com/docs/12.x/controllers.